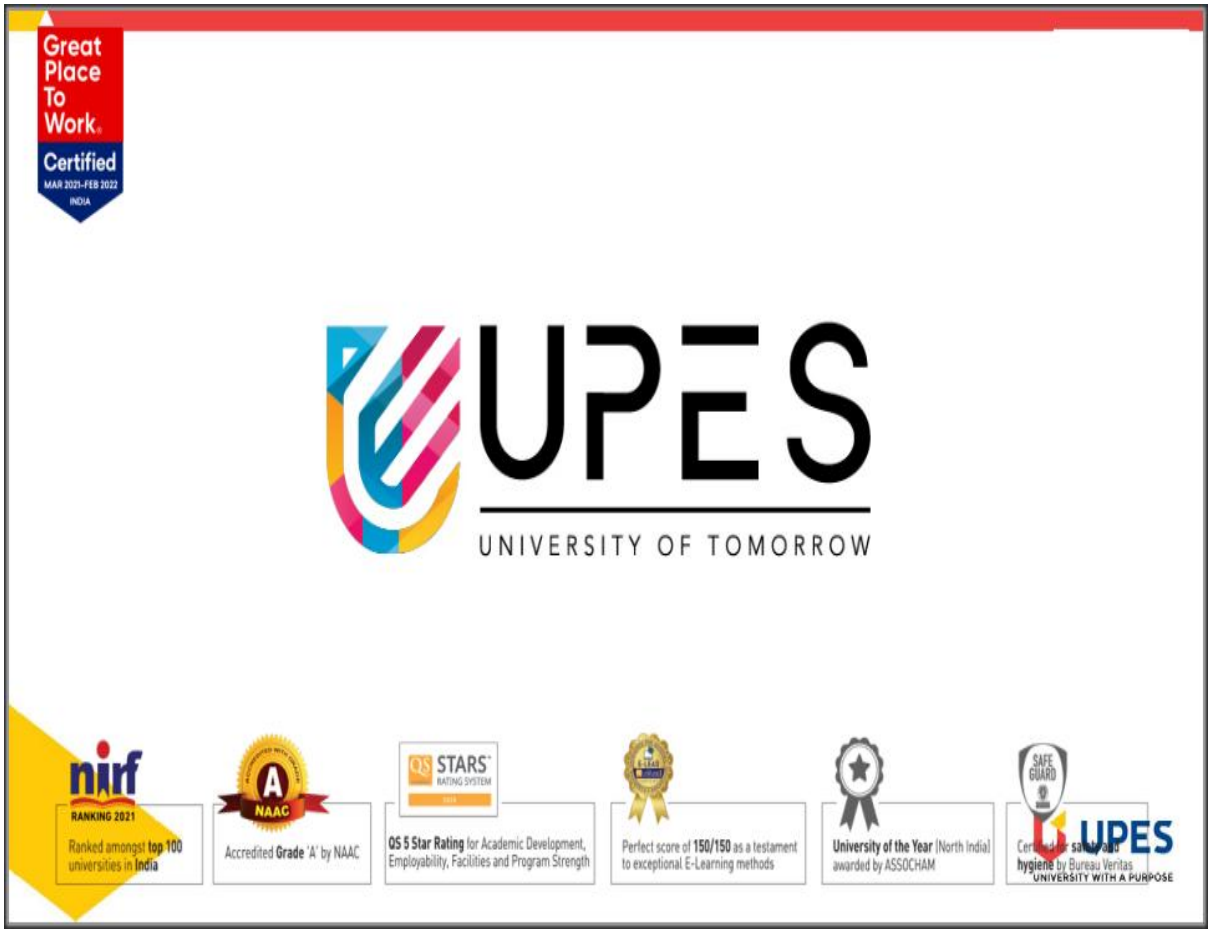
# AML LAB




Name-Anmol

Batch-20 AIML

Sap-590011794

Submitted to-Prof. Sahinur Laskar

# Index


| | |
|---|---|
| 1 | Data Processing using Python Lib |
| 2 | Predicting House Prices using linear Regression |
| 3 | Predicting Stock Prices using linear Regression |
| 4 | Customer Churn Prediction using Linear regression |
| | |
| | |
| | |
| | |

1)Data processing using Python libararies

Using pandas to plot and clean data

Using matplotlib and seaborn to plt the outputs

# 1.Team Name Normalization

1 Some team names appear in different textual formats (e.g., case differences or extra spaces). Clean the team1 and team2 columns by:

2. Converting text to lowercase
3. Removing leading/trailing spaces
4. Verifying unique team names after cleaning

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv('/content/drive/MyDrive/data.csv')
print(df)
```

```
      id  season        city        date                  team1  \
0      1    2008   Bangalore  2008-04-18  Kolkata Knight Riders
1      2    2008  Chandigarh  2008-04-19    Chennai Super Kings
2      3    2008       Delhi  2008-04-19       Rajasthan Royals
3      4    2008      Mumbai  2008-04-20         Mumbai Indians
4      5    2008     Kolkata  2008-04-20        Deccan Chargers
..   ...     ...         ...         ...                    ...
572  573    2016      Raipur  2016-05-22       Delhi Daredevils
573  574    2016   Bangalore  2016-05-24          Gujarat Lions
574  575    2016       Delhi  2016-05-25    Sunrisers Hyderabad
575  576    2016       Delhi  2016-05-27          Gujarat Lions
576  577    2016   Bangalore  2016-05-29    Sunrisers Hyderabad

                          team2                 toss_winner toss_decision  \
0    Royal Challengers Bangalore  Royal Challengers Bangalore         field
1                Kings XI Punjab          Chennai Super Kings           bat
2               Delhi Daredevils             Rajasthan Royals           bat
3    Royal Challengers Bangalore               Mumbai Indians           bat
4          Kolkata Knight Riders              Deccan Chargers           bat
..                           ...                          ...           ...
572  Royal Challengers Bangalore  Royal Challengers Bangalore         field
573  Royal Challengers Bangalore  Royal Challengers Bangalore         field
574        Kolkata Knight Riders        Kolkata Knight Riders         field
575          Sunrisers Hyderabad          Sunrisers Hyderabad         field
576  Royal Challengers Bangalore          Sunrisers Hyderabad           bat

     result  dl_applied                      winner  win_by_runs  \
0    normal           0        Kolkata Knight Riders          140
1    normal           0          Chennai Super Kings           33
2    normal           0             Delhi Daredevils            0
3    normal           0  Royal Challengers Bangalore            0
4    normal           0        Kolkata Knight Riders            0
..      ...         ...                          ...          ...
572  normal           0  Royal Challengers Bangalore            0
573  normal           0  Royal Challengers Bangalore            0
574  normal           0          Sunrisers Hyderabad           22
575  normal           0          Sunrisers Hyderabad            0
576  normal           0          Sunrisers Hyderabad            8

     win_by_wickets player_of_match  \
0                 0     BB McCullum
1                 0      MEK Hussey
2                 9     MF Maharoof
3                 5      MV Boucher
4                 5       DJ Hussey
..              ...             ...
572               6         V Kohli
573               4  AB de Villiers
574               0    MC Henriques
575               4       DA Warner
576               0     BCJ Cutting

                                         venue          umpire1  \
0                          M Chinnaswamy Stadium        Asad Rauf
1          Punjab Cricket Association Stadium, Mohali        MR Benson
2                                 Feroz Shah Kotla        Aleem Dar
3                                 Wankhede Stadium         SJ Davis
4                                     Eden Gardens        BF Bowden
```

```
df.shape
```

```
(577, 18)
```

```
print(df['team1'], df['team2'])
```

```
0      Kolkata Knight Riders
1        Chennai Super Kings
2           Rajasthan Royals
3             Mumbai Indians
4            Deccan Chargers
               ...
572         Delhi Daredevils
573            Gujarat Lions
574      Sunrisers Hyderabad
575            Gujarat Lions
576      Sunrisers Hyderabad
Name: team1, Length: 577, dtype: object 0      Royal Challengers Bangalore
1                  Kings XI Punjab
2                 Delhi Daredevils
3      Royal Challengers Bangalore
4            Kolkata Knight Riders
                  ...
572    Royal Challengers Bangalore
573    Royal Challengers Bangalore
574          Kolkata Knight Riders
575            Sunrisers Hyderabad
576    Royal Challengers Bangalore
Name: team2, Length: 577, dtype: object
```

```
df['team1']=df['team1'].str.lower().str.strip() #.str.lower() to convert into lower case and space is removed
df['team2']=df['team2'].str.lower().str.lower()
print(df['team1'], df['team2'])
```

```
0      kolkata knight riders
1        chennai super kings
2           rajasthan royals
3             mumbai indians
4            deccan chargers
               ...
572         delhi daredevils
573            gujarat lions
574      sunrisers hyderabad
575            gujarat lions
576      sunrisers hyderabad
Name: team1, Length: 577, dtype: object 0      royal challengers bangalore
1                  kings xi punjab
2                 delhi daredevils
3      royal challengers bangalore
4            kolkata knight riders
                  ...
572    royal challengers bangalore
573    royal challengers bangalore
574          kolkata knight riders
575            sunrisers hyderabad
576    royal challengers bangalore
Name: team2, Length: 577, dtype: object
```

```
print(df['team1'].unique())
print(df['team2'].unique())
```

```
['kolkata knight riders' 'chennai super kings' 'rajasthan royals'
 'mumbai indians' 'deccan chargers' 'kings xi punjab'
 'royal challengers bangalore' 'delhi daredevils' 'kochi tuskers kerala'
 'pune warriors' 'sunrisers hyderabad' 'rising pune supergiants'
 'gujarat lions']
['royal challengers bangalore' 'kings xi punjab' 'delhi daredevils'
 'kolkata knight riders' 'rajasthan royals' 'mumbai indians'
 'chennai super kings' 'deccan chargers' 'pune warriors'
 'kochi tuskers kerala' 'sunrisers hyderabad' 'rising pune supergiants'
 'gujarat lions']
```

## 2.City Name Standardization

Standardize the city column by:

1)Replacing missing city names with "Unknown"

2)Converting all city names to title case

3)Counting matches played in each city

```
print(df['city'].isnull().sum())
```

```
7
```

```
df['city'].fillna('Unknown', inplace=True)
```

```
/tmp/ipython-input-357195196.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through cha
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['city'].fillna('Unknown', inplace=True)
```

```
df['city'] = df['city'].str.title()
print(df['city'])
```

```
0       Bangalore
1      Chandigarh
2           Delhi
3          Mumbai
4         Kolkata
          ...
572        Raipur
573     Bangalore
574         Delhi
575         Delhi
576     Bangalore
Name: city, Length: 577, dtype: object
```

```
cmc=df['city'].value_counts()
print(cmc)
```

```
city
Mumbai            77
Bangalore         58
Kolkata           54
Delhi             53
Chennai           48
Chandigarh        42
Hyderabad         41
Jaipur            33
Pune              25
Durban            15
Centurion         12
Ahmedabad         12
Visakhapatnam     11
Dharamsala         9
Johannesburg       8
Unknown            7
Abu Dhabi          7
Cape Town          7
Port Elizabeth     7
Ranchi             7
Cuttack            7
Raipur             6
Sharjah            6
Rajkot             5
Kochi              5
Kimberley          3
East London        3
Nagpur             3
Bloemfontein       2
Indore             2
Kanpur             2
Name: count, dtype: int64
```

## 3. Toss Decision Text Analysis

Analyze the toss_decision column:

1)Extract unique decisions

2)Count how many times each decision was taken

3)Visualize the frequency using a bar chart

```
ud=df['toss_decision'].unique()
print(ud)
```

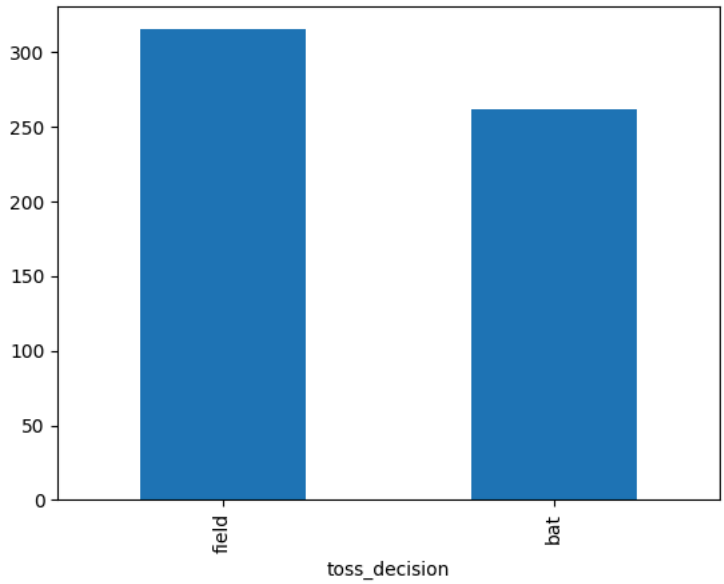
```
['field' 'bat']
```

```
dc=df['toss_decision'].value_counts()
print("Frequency of each toss decision:")
print(dc)
```

```
Frequency of each toss decision:
toss_decision
```

```
field    315
bat      262
Name: count, dtype: int64
```

```
bar=df['toss_decision'].value_counts().plot(kind='bar')
```




## 4, Winner Name Extraction

From the winner column:

1)Identify and remove rows where the match result was "No Result" or "Tie"

2)Count how many matches each team won after cleaning text values

```
df['result'].value_counts()
```

| result | count |
|---|---|
| **normal** | 568 |
| **tie** | 6 |
| **no result** | 3 |

**dtype:** int64

```
df_filtered = df[~df['result'].isin(['no result', 'tie'])]
print(df_filtered)
```

```
Original number of matches: 577
Number of matches after removing 'No Result' and 'Tie': 568
```

```
team_wins = df_filtered['winner'].value_counts()
print(team_wins)
```

```
winner
Mumbai Indians                 80
Chennai Super Kings            79
Royal Challengers Bangalore    69
Kolkata Knight Riders          68
Rajasthan Royals               61
Kings XI Punjab                61
Delhi Daredevils               56
Sunrisers Hyderabad            33
Deccan Chargers                29
Pune Warriors                  12
Gujarat Lions                   9
Kochi Tuskers Kerala            6
Rising Pune Supergiants         5
Name: count, dtype: int64
```

## 5. Player of the Match Text Frequency

Perform text analysis on player of the match:

1)Remove null values

2)Find the top 10 most frequent player names

3)Plot the results using a Seaborn bar plot

```
pn = df['player_of_match'].dropna()
print(pn)
```

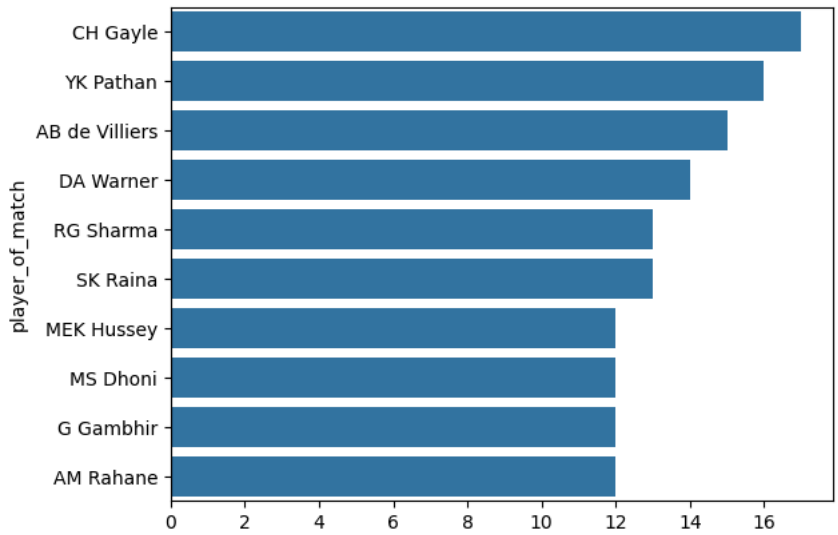
```
0         BB McCullum
1          MEK Hussey
2         MF Maharoof
3          MV Boucher
4           DJ Hussey
            ...
572           V Kohli
573    AB de Villiers
574      MC Henriques
575         DA Warner
576       BCJ Cutting
Name: player_of_match, Length: 574, dtype: object
```

```
tp=df['player_of_match'].value_counts().head(10)
print(tp)
```

```
player_of_match
CH Gayle          17
YK Pathan         16
AB de Villiers    15
DA Warner         14
RG Sharma         13
SK Raina          13
MEK Hussey        12
MS Dhoni          12
G Gambhir         12
AM Rahane         12
Name: count, dtype: int64
```

```
sns.barplot(y=tp.index, x=tp.values)
```

```
<Axes: ylabel='player_of_match'>
```




## 6. Venue Tokenization

Count how many matches were played in each venue and plot a bar chart for the top10.

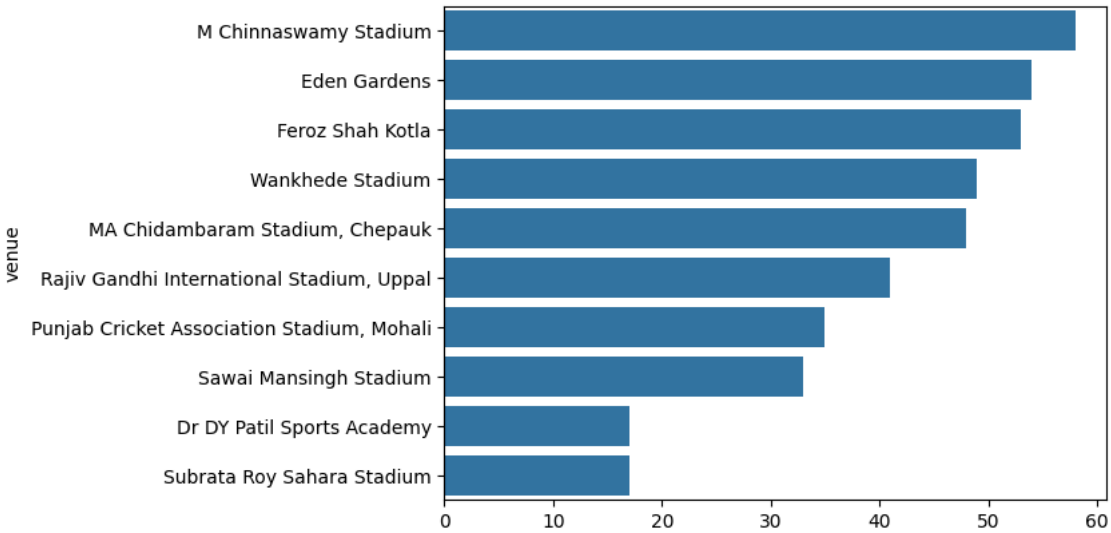
```
venue_counts = df['venue'].value_counts()
print(venue_counts.head(10))
```

```
venue
M Chinnaswamy Stadium                       58
Eden Gardens                                54
Feroz Shah Kotla                            53
```

```
Wankhede Stadium                            49
MA Chidambaram Stadium, Chepauk             48
Rajiv Gandhi International Stadium, Uppal   41
Punjab Cricket Association Stadium, Mohali  35
Sawai Mansingh Stadium                      33
Dr DY Patil Sports Academy                  17
Subrata Roy Sahara Stadium                  17
Name: count, dtype: int64
```

```
sns.barplot(y=venue_counts.head(10).index, x=venue_counts.head(10).values)
```

```
<Axes: ylabel='venue'>
```




## 7. Umpire Name Cleaning

Clean umpire columns (umpire1, umpire2, umpire3) by:

1)Replacing missing values with "Not Assigned"

2)Removing duplicate umpire names per match

3)Finding the most frequently officiating umpire

```
print(df['umpire1'].isnull().sum())
print(df['umpire2'].isnull().sum())
print(df['umpire3'].isnull().sum())
```

```
0
0
0
```

```
df['umpire3'].fillna('Not Assigned')
```

| | umpire3 |
|---|---|
| **0** | Not Assigned |
| **1** | Not Assigned |
| **2** | Not Assigned |
| **3** | Not Assigned |
| **4** | Not Assigned |
| **...** | ... |
| **572** | Not Assigned |
| **573** | Not Assigned |
| **574** | Not Assigned |
| **575** | Not Assigned |
| **576** | Not Assigned |

577 rows × 1 columns

**dtype:** object

```
print(df['umpire1'].isnull().sum())
print(df['umpire2'].isnull().sum())
print(df['umpire3'].isnull().sum())
```

```
0
0
0
```

```
umpire1=df['umpire1'].unique
umpire2=df['umpire2'].unique()
umpire3=df['umpire3'].unique()
```

```
print(df["umpire1"].head(10))
```

```
0    Asad Rauf
1    MR Benson
2    Aleem Dar
3     SJ Davis
4    BF Bowden
5    Aleem Dar
6    IL Howell
7    DJ Harper
8    Asad Rauf
9    Aleem Dar
Name: umpire1, dtype: object
```

## 8. Create a new text column match_summary by

combining:

team1, team2, winner, and season

Example: “MI vs CSK – MI won in 2019”

Display sample summaries.

```
pd.set_option('display.max_colwidth', None)
df['Match_Summary'] = df['team1'] + ' vs ' + df['team2'] + ' - ' + df['winner'].astype(str) + ' won in ' + df['season'].as
print(df['Match_Summary'].head())
```

```
0    kolkata knight riders vs royal challengers bangalore - Kolkata Knight Riders won in 2008
1                    chennai super kings vs kings xi punjab - Chennai Super Kings won in 2008
2                         rajasthan royals vs delhi daredevils - Delhi Daredevils won in 2008
3     mumbai indians vs royal challengers bangalore - Royal Challengers Bangalore won in 2008
4                deccan chargers vs kolkata knight riders - Kolkata Knight Riders won in 2008
Name: Match_Summary, dtype: object
```

## 9. Result Type Text Analysis

Analyze the result column:

Identify different textual result types

Count their occurrences

Visualize the distribution using a count plot

```
print(df['result'].unique())
```

```
['normal' 'tie' 'no result']
```

```
print(df['result'].value_counts())
```

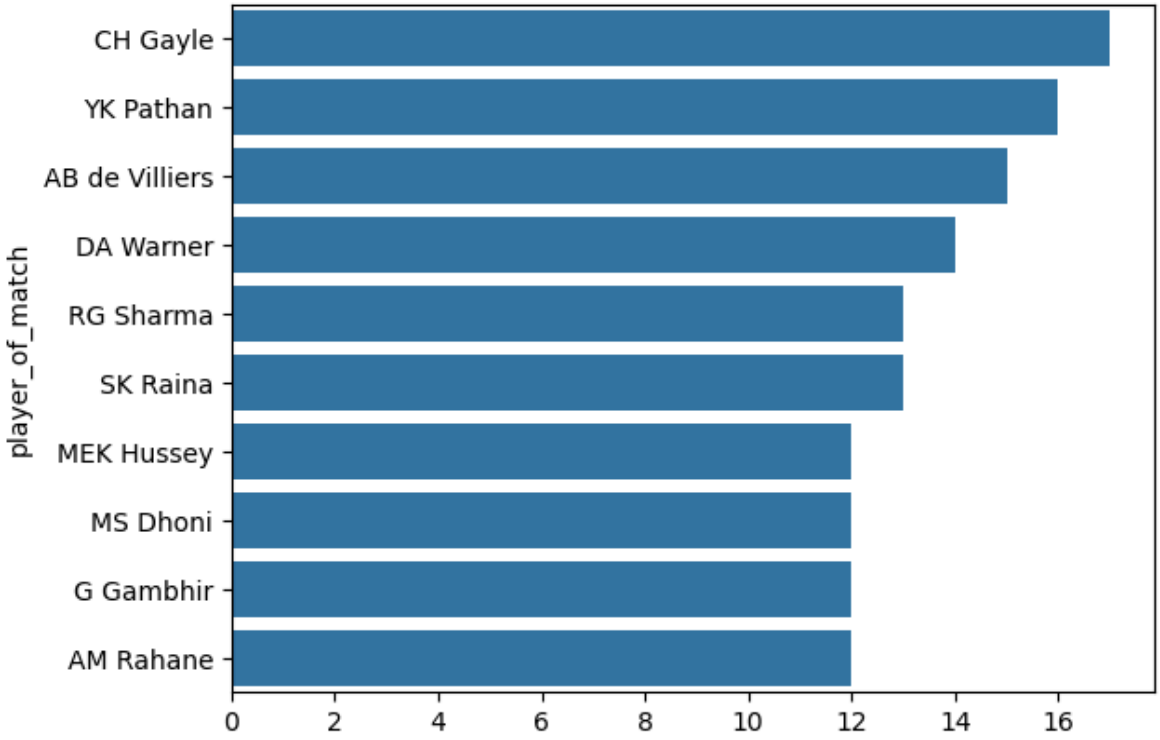
```
result
normal       568
tie            6
no result      3
Name: count, dtype: int64
```

```
sns.countplot(x=df['result'])
```

```
SK Raina          13
MEK Hussey        12
MS Dhoni          12
G Gambhir         12
AM Rahane         12
Name: count, dtype: int64
```

```
sns.barplot(y=tp.index, x=tp.values)
```

```
<Axes: ylabel='player_of_match'>
```




## 6. Venue Tokenization

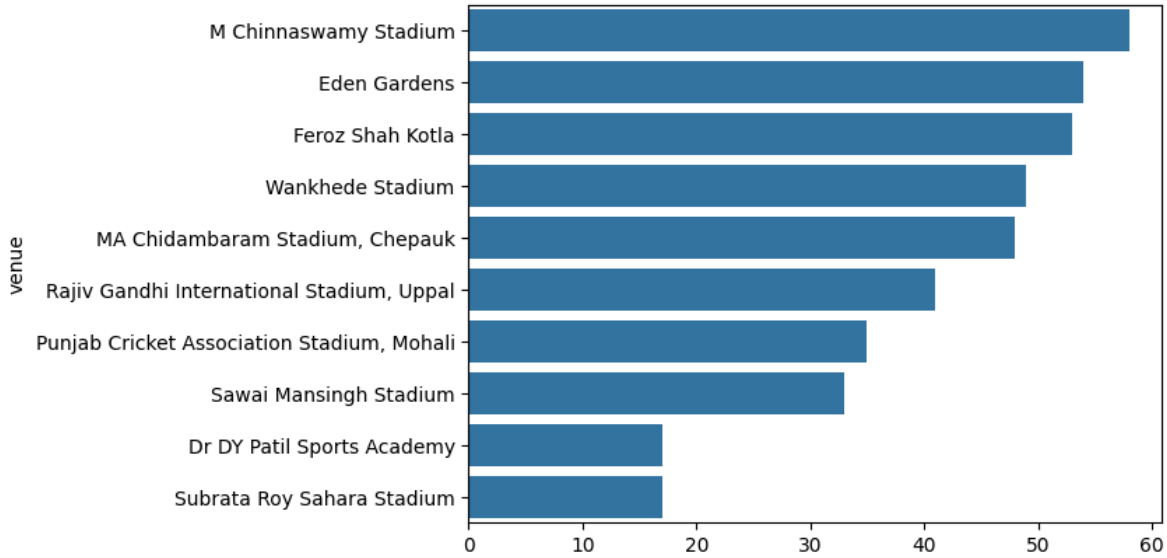
Count how many matches were played in each venue and plot a bar chart for the top10.

```
venue_counts = df['venue'].value_counts()
print(venue_counts.head(10))
```

```
venue
M Chinnaswamy Stadium                          58
Eden Gardens                                   54
Feroz Shah Kotla                               53
Wankhede Stadium                               49
MA Chidambaram Stadium, Chepauk                48
Rajiv Gandhi International Stadium, Uppal      41
Punjab Cricket Association Stadium, Mohali     35
Sawai Mansingh Stadium                         33
Dr DY Patil Sports Academy                     17
Subrata Roy Sahara Stadium                     17
Name: count, dtype: int64
```

```
sns.barplot(y=venue_counts.head(10).index, x=venue_counts.head(10).values)
```

```
<Axes: ylabel='venue'>
```




## 7. Umpire Name Cleaning

Clean umpire columns (umpire1, umpire2, umpire3) by:

1)Replacing missing values with "Not Assigned"

2)Removing duplicate umpire names per match

3)Finding the most frequently officiating umpire

```
print(df['umpire1'].isnull().sum())
print(df['umpire2'].isnull().sum())
print(df['umpire3'].isnull().sum())
```

```
0
0
0
```

```
df['umpire3'].fillna('Not Assigned')
```

| | umpire3 |
|---|---|
| **0** | Not Assigned |
| **1** | Not Assigned |
| **2** | Not Assigned |
| **3** | Not Assigned |
| **4** | Not Assigned |
| **...** | ... |
| **572** | Not Assigned |
| **573** | Not Assigned |
| **574** | Not Assigned |
| **575** | Not Assigned |
| **576** | Not Assigned |

577 rows × 1 columns

**dtype:** object

```
print(df['umpire1'].isnull().sum())
print(df['umpire2'].isnull().sum())
print(df['umpire3'].isnull().sum())
```

```
0
0
0
```

```
umpire1=df['umpire1'].unique
umpire2=df['umpire2'].unique()
umpire3=df['umpire3'].unique()
```

```
print(df["umpire1"].head(10))
```

```
0    Asad Rauf
1    MR Benson
2    Aleem Dar
3     SJ Davis
4    BF Bowden
5    Aleem Dar
6    IL Howell
7    DJ Harper
8    Asad Rauf
9    Aleem Dar
Name: umpire1, dtype: object
```

## 8. Create a new text column match_summary by

combining:

team1, team2, winner, and season

Example: “MI vs CSK – MI won in 2019”

Display sample summaries.

```
pd.set_option('display.max_colwidth', None)
df['Match_Summary'] = df['team1'] + ' vs ' + df['team2'] + ' - ' + df['winne
print(df['Match_Summary'].head())
```

```
0    kolkata knight riders vs royal challengers bangalore - Kolkata Knight Ri
1                     chennai super kings vs kings xi punjab - Chennai Super K
2                          rajasthan royals vs delhi daredevils - Delhi Darede
3     mumbai indians vs royal challengers bangalore - Royal Challengers Banga
4                deccan chargers vs kolkata knight riders - Kolkata Knight Ri
Name: Match_Summary, dtype: object
```

## 9. Result Type Text Analysis

Analyze the result column:

Identify different textual result types

Count their occurrences

Visualize the distribution using a count plot

```
print(df['result'].unique())
```

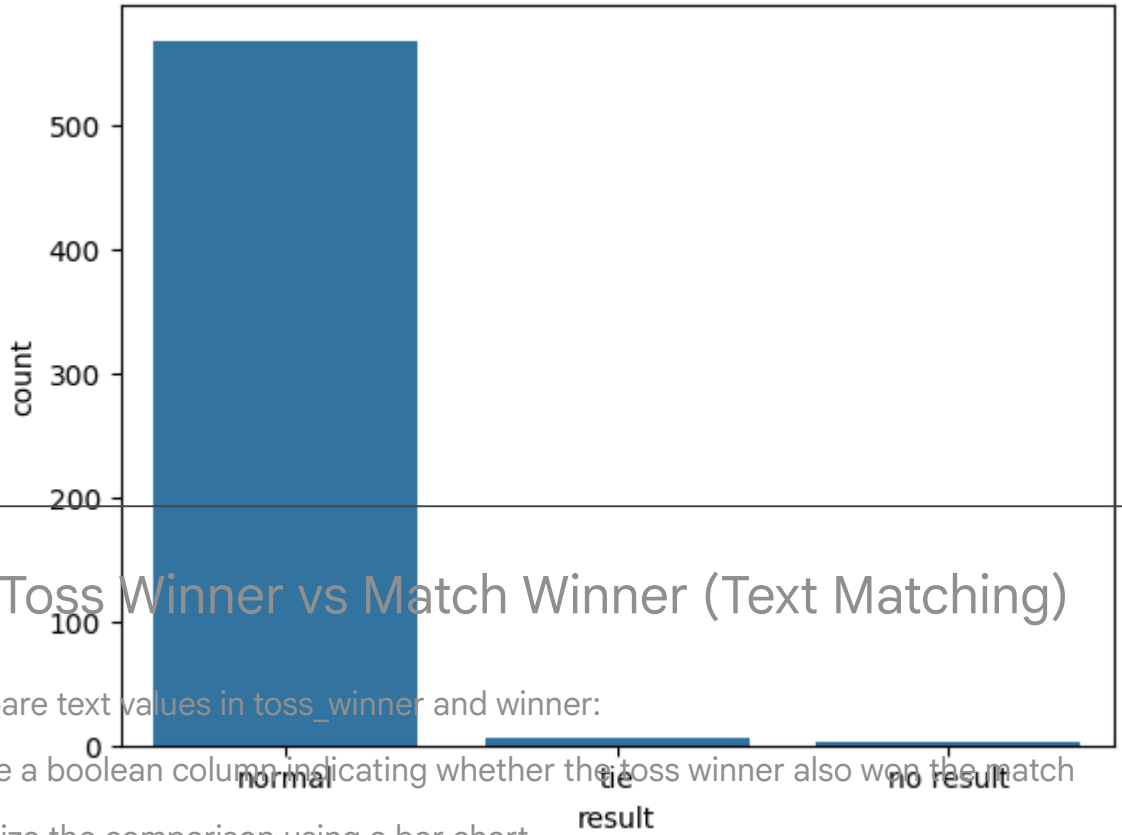
```
['normal' 'tie' 'no result']
```

```
print(df['result'].value_counts())
```

```
result
normal       568
tie            6
no result      3
Name: count, dtype: int64
```

```
sns.countplot(x=df['result'])
```

```
<Axes: xlabel='result', ylabel='count'>
```




## 10. Toss Winner vs Match Winner (Text Matching)

Compare text values in toss_winner and winner:

Create a boolean column indicating whether the toss winner also won the match

Visualize the comparison using a bar chart

```
df['toss_decision']=df['toss_winner'].str.strip().str.lower()
df['winner']=df['winner'].str.strip().str.lower()
df['toss_won']=df['toss_winner']==df['winner']
print(df['toss_won'].value_counts())
```

```
toss_won
False    577
Name: count, dtype: int64
```

```
sns.barplot(x=df['toss_won'].value_counts().index, y=df['toss_won'].value_cou
```

```
<Axes: xlabel='toss_won'>
```

# EXE-2

## Step-0 Data Processing

Python libraries such as pandas, matplotlib, and seaborn were used to handle and visualize the dataset. Pandas was used for loading, inspecting, and cleaning the data, while matplotlib and seaborn were used to generate plots that help analyze relationships between variables and understand patterns in the dataset.

## Step-1 Problem Defination

The objective of this experiment is to build a machine learning model that can predict house prices using the given dataset. The dataset contains features such as area, number of bedrooms, location, and other property attributes. Since price is a continuous numeric value, this problem is a regression task. The model must learn how these features influence price and estimate it accurately.

## Step-2 Data understanding

Using functions like df.head() and df.info(), the dataset structure was examined. Each row represents a house, and each column represents a feature describing that house.

The dataset contains:

Numerical features : Area, Bedrooms, Bathrooms, Age, Parking, Balcony, etc.

Categorical features : Location and Furnished

The target variable is Price, which is denoted as y, and the remaining features are used as input variables x. Since machine learning models require numeric input, categorical features must be encoded before training.

**Step-3 Data Processing**

Categorical variables such as Location and Furnished were converted into numerical format using one-hot encoding. This creates new columns such as:

- Location_Urban
- Location_Rural
- Furnished_Yes

This transformation was done using the get_dummies() function. Numerical features were already in usable format and therefore did not require modification. This preprocessing step ensures the dataset is suitable for machine learning algorithms. This aligns with the preprocessing stage of the pipeline where raw data is transformed into model-ready numerical form

**Step-4 Data Selection and Model Training**

As discussed earlier, Price was selected as the target variable (y), and all other columns were selected as input features (x). For demonstration purposes, the model was trained on the full dataset instead of splitting into training and testing sets.

The model chosen was Linear Regression, since the dependent variable is continuous. Regression models are specifically designed to predict numeric outputs, unlike classification models which predict categories.

**Step 5-Prediction**

After training, the model was used to generate predictions using the .predict() function. By providing input feature values (X), the model calculates estimated house prices based

on the learned relationship between features and target variable.

**Step-6 Evaluation**

The model performance was evaluated using:

Mean Squared Error (MSE) -- measures average squared difference between predicted and actual prices

R^2 Score - measures how well the model explains variation in price .

The obtained values showed MSE close to zero and $R^2$ close to 1, indicating that the model fit the training data extremely well. According to regression evaluation theory, lower MSE

Additionally, the results were visualized using a scatter plot to compare predicted prices with actual prices, allowing visual inspection of model accuracy.

## Predicting Housing Prices: Develop a regression model to predict house prices based on features like location, size, and amenities. italicized text

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import linear_model as lm
from sklearn.metrics import mean_squared_error, r2_score
```

```
df=pd.read_csv('/content/drive/MyDrive/Housing_Price_Dataset_120_Records.csv')
print(df.head())
```

```
  HouseID  Location  Area_sqft  Bedrooms  Bathrooms  Balcony  Parking  \
0    H001     Rural       1028         1          3        0        0
1    H002     Rural       2316         1          4        0        0
2    H003     Urban       1834         1          2        2        2
3    H004     Urban       1719         3          1        0        2
4    H005  Suburban       1118         2          3        0        0

   Age_Years Furnished   Price
0          5       Yes   55.84
1          3       Yes  103.88
2         23        No   86.42
3         14        No   96.77
4         13       Yes   63.94
```

Data Preprocessing

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 120 entries, 0 to 119
Data columns (total 10 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   HouseID    120 non-null    object
 1   Location   120 non-null    object
 2   Area_sqft  120 non-null    int64
 3   Bedrooms   120 non-null    int64
 4   Bathrooms  120 non-null    int64
 5   Balcony    120 non-null    int64
 6   Parking    120 non-null    int64
 7   Age_Years  120 non-null    int64
 8   Furnished  120 non-null    object
 9   Price      120 non-null    float64
dtypes: float64(1), int64(6), object(3)
memory usage: 9.5+ KB
```

```
pd.Index(['HouseID','Location','Area_sqft','Bedrooms','Bathrooms','Balcony','Parking','Age_Years','Furnished','Price'])
```

```
Index(['HouseID', 'Location', 'Area_sqft', 'Bedrooms', 'Bathrooms', 'Balcony',
       'Parking', 'Age_Years', 'Furnished', 'Price'],
      dtype='object')
```

```
df=pd.get_dummies(df,columns=['Furnished','Location'],drop_first=False)
```

```
df.head()
```

| | HouseID | Area_sqft | Bedrooms | Bathrooms | Balcony | Parking | Age_Years | Price | Furnished_No | Furnished_Yes | Location_Rural |
|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | H001 | 1028 | 1 | 3 | 0 | 0 | 5 | 55.84 | False | True | True |
| **1** | H002 | 2316 | 1 | 4 | 0 | 0 | 3 | 103.88 | False | True | True |
| **2** | H003 | 1834 | 1 | 2 | 2 | 2 | 23 | 86.42 | True | False | False |
| **3** | H004 | 1719 | 3 | 1 | 0 | 2 | 14 | 96.77 | True | False | False |
| **4** | H005 | 1118 | 2 | 3 | 0 | 0 | 13 | 63.94 | False | True | False |

```
reg=lm.LinearRegression()
reg.fit(df[['Area_sqft','Bedrooms','Bathrooms','Balcony','Parking','Age_Years','Furnished_Yes','Location_Suburban','Locatio
```

```
▾ LinearRegression
LinearRegression()
```

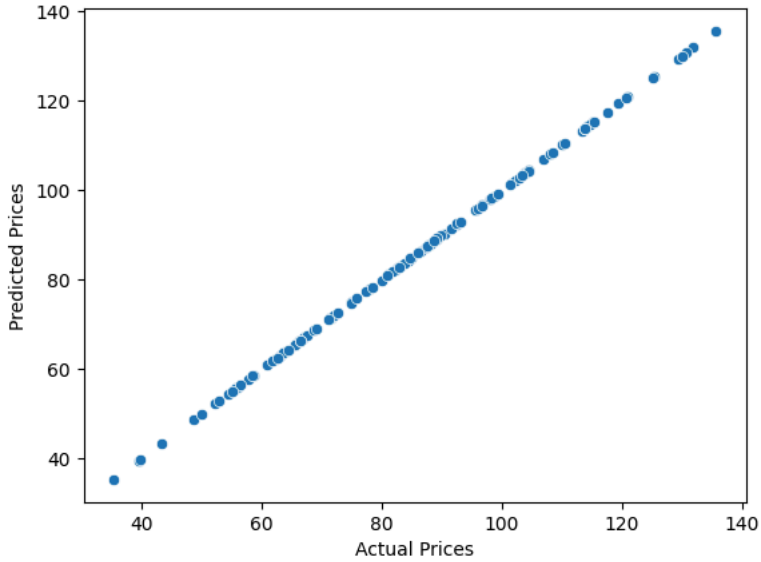
```
import pickle
with open('model_pickle','wb') as f:
  pickle.dump(reg,f)
```

```
y_pred = reg.predict(df[['Area_sqft','Bedrooms','Bathrooms','Balcony','Parking','Age_Years','Furnished_Yes','Location_Subur
mse = mean_squared_error(df.Price, y_pred)
r2 = r2_score(df.Price, y_pred)
print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')
```

```
Mean Squared Error: 6.344966032873642e-27
R^2 Score: 1.0
```

```
plot = sns.scatterplot(x=df.Price, y=y_pred)
plot.set_xlabel('Actual Prices')
plot.set_ylabel('Predicted Prices')
```

```
Text(0, 0.5, 'Predicted Prices')
```



# EXE-3

## Step-0 Libraries Used

Several Python libraries were used to perform data handling, model training, evaluation, and visualization for stock price prediction.

**NumPy:** was used for numerical operations and array manipulation required during model input preparation.

**Pandas:** was used for loading the dataset, filtering stock data, converting date formats, and creating new features such as the numerical time variable (Days).

**Matplotlib** and **Seaborn:** were used for visualization to plot stock price trends and analyze the distribution of values.

**Scikit-learn:** was used to implement machine learning functionality. The train_test_split module was used to divide data into training and testing sets, the LinearRegression model was used for prediction, and evaluation metrics such as **Mean Squared Error** and **$R^2$ score** were used to assess model performance.

**Pickle:** was used to save the trained model so it can be reused later without retraining.

## Step-1 Problem Definition

The objective of this experiment is to develop a time series prediction model to forecast stock prices. The dataset contains historical stock data including Date, Open, High, Low, Close prices, and Volume for different companies. Since the output variable (stock price) is numeric and changes over time, this is a regression-based time series prediction problem.

## Step-2 Data Understanding

The dataset was loaded using pandas and inspected to understand its structure. Each row represents a stock record for a specific date, and each column represents a feature describing that day's market performance.

The dataset contains the following attributes:

- Date → trading day
- Stock → company ticker symbol
- Open, High, Low, Close → price values
- Volume → number of shares traded

As shown in the dataset, the data contains multiple stocks such as AAPL and TSLA ,For modeling purposes, only one stock (AAPL) was selected so that a single time series could be analyzed.

**Step-3 Data Preprocessing**

The Date column was converted into datetime format to enable time-based calculations. A new feature called Days was created by subtracting the minimum date from each date value. This converts time into numerical form representing the number of days since the start of the dataset.

This step is necessary because machine learning models cannot directly process date values; they require numerical inputs.

**Step-4 Feature Selection, Model Selection and Training**

In this step, the input (X) was taken as the Days column, which represents time in numeric form, and the output (y) was selected as the closing stock price.

The dataset was divided into training and testing sets using an 80:20 ratio. Shuffling was disabled so that the chronological

order of dates remains unchanged. This is important for time-series data because predictions should always be made using past values to estimate future ones.

A Linear Regression model was chosen for this task because the target variable is continuous. The model was trained using the training data, where it learned the relationship between time and stock price by fitting a line that minimizes prediction error. After training, the model parameters were stored so that it could later be reused without retraining.

**Step-6 Prediction**

After training, the model was used to predict stock prices for the test dataset. The predictions generated represent estimated closing prices based solely on time progression.

As shown in the output, the predicted values form a continuous decreasing trend, indicating the model learned a linear relationship

Step-7 Evaluation

Model performance was evaluated using two regression metrics:

- Mean Squared Error (MSE):Measures average squared difference between predicted and actual values. Lower values indicate better performance.

- $R^2$ Score:Measures how well the model explains variance in the data. Values closer to 1 indicate better fit.

From the results :

MSE is approx.= 3072

$R^2$ is approx.= $-11.21$

The negative $R^2$ indicates the model performs poorly and does not fit the data well. This occurs because stock prices follow nonlinear patterns, while linear regression can only model straight-line relationships.

## Predicting Stock Prices: Develop a time series prediction model to forecast stock prices.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import linear_model as lm
from sklearn.metrics import r2_score,mean_squared_error
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/stock_price_timeseries_dataset.csv')
df
```

| | Date | Stock | Open | High | Low | Close | Volume |
|---|---|---|---|---|---|---|---|
| **0** | 2020-01-01 | AAPL | 173.80 | 174.75 | 173.68 | 174.43 | 8204212 |
| **1** | 2020-01-02 | AAPL | 176.01 | 177.59 | 174.07 | 177.55 | 2766891 |
| **2** | 2020-01-03 | AAPL | 177.57 | 178.37 | 176.62 | 176.71 | 5721339 |
| **3** | 2020-01-06 | AAPL | 176.01 | 177.58 | 171.33 | 171.73 | 9242680 |
| **4** | 2020-01-07 | AAPL | 171.85 | 172.20 | 170.69 | 170.82 | 4416664 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **6520** | 2024-12-25 | TSLA | 225.09 | 225.21 | 221.66 | 223.18 | 9701165 |
| **6521** | 2024-12-26 | TSLA | 222.07 | 222.87 | 220.31 | 220.32 | 6045143 |
| **6522** | 2024-12-27 | TSLA | 219.94 | 221.69 | 218.38 | 219.57 | 8512677 |
| **6523** | 2024-12-30 | TSLA | 219.54 | 220.33 | 216.70 | 217.33 | 9137479 |
| **6524** | 2024-12-31 | TSLA | 216.47 | 219.61 | 216.19 | 218.41 | 2097124 |

6525 rows × 7 columns

Next steps: Generate code with df  New interactive sheet

```
df_stock=df[df['Stock']=='AAPL']
df_stock.head()
```

| | Date | Stock | Open | High | Low | Close | Volume |
|---|---|---|---|---|---|---|---|
| **0** | 2020-01-01 | AAPL | 173.80 | 174.75 | 173.68 | 174.43 | 8204212 |
| **1** | 2020-01-02 | AAPL | 176.01 | 177.59 | 174.07 | 177.55 | 2766891 |
| **2** | 2020-01-03 | AAPL | 177.57 | 178.37 | 176.62 | 176.71 | 5721339 |
| **3** | 2020-01-06 | AAPL | 176.01 | 177.58 | 171.33 | 171.73 | 9242680 |
| **4** | 2020-01-07 | AAPL | 171.85 | 172.20 | 170.69 | 170.82 | 4416664 |

Next steps: Generate code with df_stock  New interactive sheet

```
df_stock['Date']=pd.to_datetime(df_stock['Date'])
df_stock.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1305 entries, 0 to 1304
Data columns (total 7 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    1305 non-null   datetime64[ns]
 1   Stock   1305 non-null   object
 2   Open    1305 non-null   float64
 3   High    1305 non-null   float64
 4   Low     1305 non-null   float64
 5   Close   1305 non-null   float64
 6   Volume  1305 non-null   int64
dtypes: datetime64[ns](1), float64(4), int64(1), object(1)
memory usage: 113.9+ KB
```

```
/tmp/ipython-input-31740583.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  df_stock['Date']=pd.to_datetime(df_stock['Date'])
```

```
df_stock['Days']=(df_stock['Date']-df_stock['Date'].min())
df_stock
```

```
/tmp/ipython-input-1482232864.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  df_stock['Days']=(df_stock['Date']-df_stock['Date'].min())
```

| | Date | Stock | Open | High | Low | Close | Volume | Days |
|---|---|---|---|---|---|---|---|---|
| **0** | 2020-01-01 | AAPL | 173.80 | 174.75 | 173.68 | 174.43 | 8204212 | 0 days |
| **1** | 2020-01-02 | AAPL | 176.01 | 177.59 | 174.07 | 177.55 | 2766891 | 1 days |
| **2** | 2020-01-03 | AAPL | 177.57 | 178.37 | 176.62 | 176.71 | 5721339 | 2 days |
| **3** | 2020-01-06 | AAPL | 176.01 | 177.58 | 171.33 | 171.73 | 9242680 | 5 days |
| **4** | 2020-01-07 | AAPL | 171.85 | 172.20 | 170.69 | 170.82 | 4416664 | 6 days |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... |
| **1300** | 2024-12-25 | AAPL | 207.96 | 210.35 | 206.18 | 208.42 | 4510463 | 1820 days |
| **1301** | 2024-12-26 | AAPL | 208.69 | 214.63 | 207.23 | 213.99 | 4335705 | 1821 days |
| **1302** | 2024-12-27 | AAPL | 213.99 | 217.12 | 212.77 | 216.24 | 2601207 | 1822 days |
| **1303** | 2024-12-30 | AAPL | 216.62 | 217.17 | 213.72 | 215.53 | 7203690 | 1825 days |
| **1304** | 2024-12-31 | AAPL | 215.26 | 218.50 | 215.06 | 217.00 | 5083372 | 1826 days |

1305 rows × 8 columns

Next steps: Generate code with df_stock New interactive sheet

```
x=df_stock['Days']
y=df_stock['Close']
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,shuffle=False)
```

```
model=lm.LinearRegression()
model.fit(x_train.dt.days.values.reshape(-1,1), y_train)
with open('model_pickle','wb') as f:
  pickle.dump(model,f)
```

```
y_pred=model.predict(x_test.dt.days.values.reshape(-1,1))
y_pred
```

```
       150.49573228, 150.47262862, 150.40331765, 150.380214  ,
       150.35711034, 150.33400669, 150.31090303, 150.24159206,
       150.21848841, 150.19538475, 150.17228109, 150.14917744,
       150.07986647, 150.05676281, 150.03365916, 150.0105555 ,
       149.98745185, 149.91814088, 149.89503722, 149.87193357,
       149.84882991, 149.82572625, 149.75641529, 149.73331163,
       149.71020797, 149.68710432, 149.66400066, 149.59468969,
       149.57158604, 149.54848238, 149.52537873, 149.50227507,
       149.4329641 , 149.40986045, 149.38675679, 149.36365313,
       149.34054948, 149.27123851, 149.24813485, 149.2250312 ,
       149.20192754, 149.17882389, 149.10951292, 149.08640926,
       149.06330561, 149.04020195, 149.01709829, 148.94778733,
       148.92468367, 148.90158001, 148.87847636, 148.8553727 ,
       148.78606173, 148.76295808, 148.73985442, 148.71675077,
       148.69364711, 148.62433614, 148.60123249, 148.57812883,
       148.55502517, 148.53192152, 148.46261055, 148.43950689,
       148.41640324, 148.39329958, 148.37019593, 148.30088496,
       148.2777813 , 148.25467765, 148.23157399, 148.20847033,
       148.13915937, 148.11605571, 148.09295205, 148.0698484 ,
       148.04674474, 147.97743377, 147.95433012, 147.93122646,
```

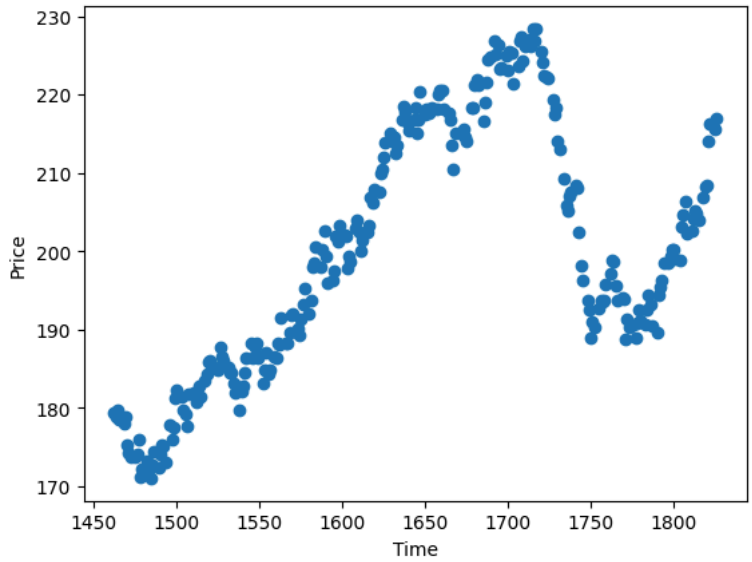
```
       146.84353463, 146.82223097, 146.79914752, 146.77604566,
       146.75294001, 146.68362904, 146.66052538, 146.63742173,
       146.61431807, 146.59121441, 146.52190345, 146.49879979,
       146.47569613, 146.45259248, 146.42948882, 146.36017785,
       146.3370742 , 146.31397054, 146.29086689, 146.26776323,
       146.19845226, 146.17534861, 146.15224495, 146.12914129,
       146.10603764, 146.03672667, 146.01362301, 145.99051936,
       145.9674157 , 145.94431205, 145.87500108, 145.85189742,
       145.82879377, 145.80569011, 145.78258645, 145.71327549,
       145.69017183, 145.66706817, 145.64396452, 145.62086086,
       145.55154989, 145.52844624, 145.50534258, 145.48223893,
       145.45913527, 145.3898243 , 145.36672065, 145.34361699,
       145.32051333, 145.29740968, 145.22809871, 145.20499505,
       145.1818914 , 145.15878774, 145.13568409, 145.06637312,
       145.04326946, 145.02016581, 144.99706215, 144.97395849,
       144.90464753, 144.88154387, 144.85844021, 144.83533656,
       144.8122329 , 144.74292193, 144.71981828, 144.69671462,
       144.67361097, 144.65050731, 144.58119634, 144.55809269,
       144.53498903, 144.51188537, 144.48878172, 144.41947075,
       144.39636709, 144.37326344, 144.35015978, 144.32705613,
       144.25774516, 144.2346415 , 144.21153785, 144.18843419,
       144.16533053, 144.09601957, 144.07291591, 144.04981225,
       144.0267086 , 144.00360494, 143.93429397, 143.91119032,
       143.88808666, 143.86498301, 143.84187935, 143.77256838,
       143.74946473, 143.72636107, 143.70325741, 143.68015376,
       143.61084279, 143.58773913, 143.56463548, 143.54153182,
       143.51842817, 143.4491172 , 143.42601354, 143.40290989,
       143.37980623, 143.35670257, 143.28739161, 143.26428795,
       143.24118429, 143.21808064, 143.19497698, 143.12566601,
       143.10256236])
```

```
mse=mean_squared_error(y_test,y_pred)
mse
```

```
3072.378496877529
```

```
r2=r2_score(y_test,y_pred)
r2
```

```
-11.213999810715315
```

```
plt.scatter(x_test.dt.days,y_test)
plt.xlabel('Time')
plt.ylabel('Price')
```

```
Text(0, 0.5, 'Price')
```